

STREAMORA

Premium Movie Streaming Platform — Project Requirements Document

Important data-source clarification: Movies must be fetched from a free online movie API. JSON Server must not be used as the movie catalogue or movie-data source. JSON Server or IndexedDB/db.js may be used only for local application data such as user accounts, favourites, subscriptions, watch history, and admin-managed records.

1. Project Overview

Students will build a modern, responsive movie-streaming web application using React and Tailwind CSS. The platform allows visitors to discover movies from a free online API, create accounts, subscribe through Flutterwave test mode, save favourite movies, and access available movie videos or trailers.

2. Required Technology

Technology	Requirement
React	Component-based application development
Tailwind CSS	Modern, responsive interface design
Free movie API	Required source for movie catalogue, search, details, posters, genres, and ratings where available
JSON Server / IndexedDB / db.js	Optional local persistence for app-specific data only; not for fetching movies
Flutterwave	Test-mode subscription payment flow only
React Router	Public, protected, and admin routes
Motion / Framer Motion or equivalent	Use for polished transitions and animations, especially modals, dialogs, page transitions, and interactive UI states
Swiper or equivalent	Use Swiper for sliding carousels such as featured movies, popular titles, categories, and top picks

3. UI, Animation, and Interaction Requirements

The application should feel modern, polished, and production-ready. Students are expected to use animation and interaction libraries where appropriate rather than relying only on static UI elements.

- **Modal animations:** Use Motion, Framer Motion, Animation Presence, or another suitable animation package to animate modal/dialog entrance and exit, including fade, scale, or slide transitions.
- **Interactive feedback:** Add subtle hover, focus, tap, loading, and state-change animations where they improve usability without becoming distracting.
- **Carousels:** Use **Swiper** or another suitable carousel package for sliding movie sections. Carousels should support responsive breakpoints and appropriate navigation controls.
- **Responsive behavior:** Animations, modals, and carousels must work properly on mobile, tablet, laptop, and desktop screen sizes.
- **Accessibility:** Interactive elements must remain keyboard-accessible, and animations should not prevent users from understanding or using the interface.
- **Performance:** Avoid excessive animations. Use animation libraries responsibly and ensure that transitions do not noticeably slow down the application.

4. Free Movie API Requirement

Mandatory: The movie catalogue must be loaded from a free online API at runtime. Students may use a suitable free service such as TMDB, OMDb, TVMaze where appropriate, or another approved free movie API.

- Fetching movie lists from the API using fetch or an equivalent method.
- Displaying loading/skeleton states while API requests are in progress.
- Displaying useful error messages when the API request fails.
- Searching or filtering movies using API data or supported query parameters.
- Opening a movie-details page using the selected movie's API ID.
- Displaying posters, titles, release dates, genres, ratings, and descriptions where supplied by the API.
- Handling empty API results gracefully.
- Keeping API keys in environment variables when the selected service requires a key.

Important: TMDB provides movie metadata and images but does not generally provide unrestricted full-movie streaming. Students must use legally available trailers, public-domain films, Creative Commons videos, or another authorized video source for playback. They must not scrape or embed unauthorized copyrighted streams.

5. Main User Pages

Page	Required functionality
Home	Hero section, featured movies from the free API, categories/genres, search, responsive movie cards, and suitable animated/carousel sections.
Browse Movies	Movie catalogue fetched from the API, filtering, sorting where possible, pagination or load more.
Movie Details	Poster, title, synopsis, rating, release date, genre, trailer/video area, favourite button, and subscription prompt.
Register	Create a user account with validation.
Login	Authenticate users and display appropriate feedback.
Favourites	Display movies saved by the logged-in user.
Subscription Plans	Show available plans, benefits, and subscription action.
Payment	Show payment summary and redirect to Flutterwave test checkout.
Thank You / Payment Success	Confirm successful test payment and show next steps.
Profile	Display account details, subscription status, and watch history where implemented.

6. Admin Pages

- **Admin Login:** Separate login interface with protected access.
- **Admin Dashboard:** Summary cards for users, subscriptions, payments, favourites, and catalogue activity.
- **User Management:** View users, inspect account status, and activate/deactivate accounts in the local data layer.
- **Content Management:** Manage featured selections, categories, promotional banners, or approved video links. The main movie catalogue must still come from the free movie API.
- **Subscriptions/Payments:** View test subscription records, payment references, statuses, and dates.

7. Core Functional Requirements

- Users can register and log in.
- Users can browse movies fetched from the free API.
- Users can search for movies and view details.
- Users can add and remove movies from favourites.
- Only users with an active subscription can access the platform's protected viewing experience.
- The payment flow must use Flutterwave test mode only. No real payment is required.
- Successful test payments must lead to a Thank You / Payment Success page.
- The interface must include skeleton loaders, spinners, empty states, and error states.
- The app must be responsive on mobile, tablet, laptop, and large desktop screens.
- Admin routes must be protected and accessible only to admin users.
- Modals and important interactive transitions should use an animation library such as Motion, Framer Motion, or an equivalent package.
- Sliding carousels should use Swiper or an equivalent package and must be responsive.

8. Suggested Local Data Structure

If JSON Server or IndexedDB/db.js is used, it should store application-specific records such as:

- users
- favourites
- subscriptions
- payments
- watchHistory
- adminSettings
- featuredMovieIds or approvedVideoLinks

Do not copy the full movie catalogue into JSON Server. Movie information must be requested from the free online API.

9. Assessment Rubric — 100 Marks

Area	Marks
Free movie API integration and correct data fetching	20
React structure, reusable components, and routing	15

Responsive Tailwind CSS design	15
Authentication and protected routes	10
Favourites and subscription logic	10
Flutterwave test payment and success page	10
Admin pages and role-based access	10
Loading, error, and empty states	5
Code quality, organization, animation, carousel use, and user experience	5
Total	100

10. Submission Expectations

- Submit the complete React project with clear folder organization.
- Include a README explaining the chosen free movie API and setup steps.
- Use an .env file for API keys where required, and provide an example environment file without secret values.
- Include screenshots or a short demonstration of the main user and admin flows.
- Ensure the project runs locally without relying on a pre-populated movie collection in JSON Server.
- Demonstrate at least one animated modal and one responsive sliding carousel in the submitted project.